

CLASS 2

Build the Registration API with DRF

Connect every idea to our CRM project.

POINT 1

What is DRF?

Django REST Framework is a Django package that helps us build APIs.

Serializer

Converts and validates.

ViewSet

Controls API actions.

Router

Creates API URLs.

Browsable API

Tests in browser.

```
crm/crm/settings.py -> 'rest_framework'
```

CODE CONNECTION

Where is the DRF code?

Concept	Project file
Activate DRF	crm/crm/settings.py
Serializer	crm/registration/serializers.py
ViewSet	crm/registration/views.py
Router	crm/registration/urls.py

Open each file while explaining the matching concept.

POINT 2

What is CRUD?

Create

Add a registration.

Read

See registrations.

Update

Change a registration.

Delete

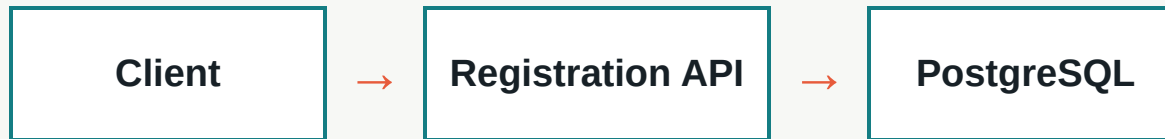
Remove a registration.

CRUD means the four basic operations we perform on database data.

POINT 3

What is an API?

An API lets another software request and use our Registration data.



```
/account/api/registrations/
```

POINT 4

What is JSON?

JSON is the structured data format our API sends and receives.

```
{  
  "id": 5,  
  "name": "Student",  
  "email": "student@example.com",  
  "date_of_birth": "2001-02-03"  
}
```

It looks similar to a Python dictionary, but JSON is a data exchange format.

POINT 5

What is a REST API?

It uses HTTP methods and predictable URLs to work with data.

GET	/account/api/registrations/	-> Read all
POST	/account/api/registrations/	-> Create
GET	/account/api/registrations/5/	-> Read one
PATCH	/account/api/registrations/5/	-> Update part
DELETE	/account/api/registrations/5/	-> Delete

POINT 6

Why use Django REST Framework?

Django can build APIs manually. DRF gives us ready tools, so we write less repeated code.



POINT 7

What changed in our project?

Before

HTML form → Django view → Model → PostgreSQL
→ HTML page

After

API request → ViewSet → Serializer → Model →
PostgreSQL → JSON

Same database. A new way to send and receive Registration data.

POINT 8

Serializer: converter and validator

```
class RegistrationSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Registration  
        fields = ['id', 'name', 'email',  
                  'password', 'date_of_birth']
```

Outgoing: model object → JSON-ready data

Incoming: JSON → validated model-ready data

POINT 9

ViewSet: the API controller

```
class RegistrationViewSet(viewsets.ModelViewSet):  
    queryset = Registration.objects.all()  
    serializer_class = RegistrationSerializer
```

queryset: which database data to use.

serializer_class: how to convert and validate that data.

It controls list, create, retrieve, update, and delete.

VIEWSET EXAMPLE

What happens during an update?



The ViewSet controls the action; the Serializer checks the data.

POINT 10

Router: automatic URL maker

```
router = DefaultRouter()
router.register('registrations', v.RegistrationViewSet)

path('api/', include(router.urls))
```

This one registration creates list, create, detail, update, and delete URLs.

Router connects URLs to the correct ViewSet actions.

POINT 11

Browsable API: test from the browser

DRF gives us a ready testing page with:

GET

View data.

POST

Create data.

PUT/PATCH

Update data.

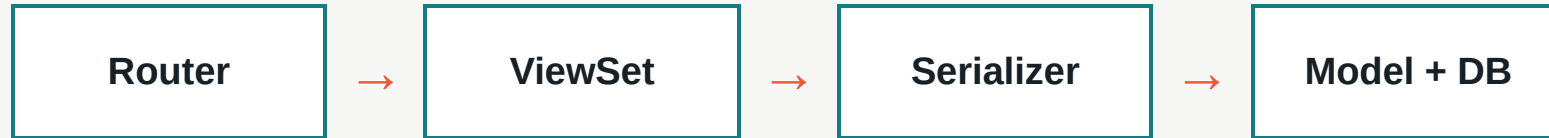
DELETE

Delete data.

```
http://127.0.0.1:8001/account/api/registrations/
```

FINAL MENTAL MODEL

Remember this flow



Model: database design **Serializer:** converter/checker

ViewSet: controller **Router:** URL maker